

Riassunto Ingegneria del Software

Angelo Vaccaro

July 2026

1 Fondamenti di Ingegneria del Software

1.1 La Complessità del Software e la 'Crisi'

Sviluppare un sistema software di buona qualità è un'operazione estremamente complessa, definita come *programming-in-the-large*, che coinvolge molte persone per lunghi periodi e produce milioni di righe di codice. Questo vale sia per i sistemi informativi aziendali, sia per i sistemi *real-time* ed *embedded* soggetti a rigidi vincoli di tempo e prestazioni.

Secondo i dati di mercato, una grande percentuale di progetti IT fallisce o incontra gravi difficoltà, manifestando ritardi, sfioramento dei costi previsti (*over budget*) o mancanza di funzionalità. Le cause principali di questi fallimenti non sono quasi mai di natura tecnologica. I problemi derivano invece da natura umana e organizzativa: difficoltà di comunicazione con gli *stakeholder* (i portatori di interesse), incomprensione dei veri bisogni del cliente, cattiva gestione manageriale e difficoltà nel comprendere il “dominio del business”, ovvero il contesto operativo in cui il software si andrà a inserire. Esempi storici di fallimenti disastrosi, dovuti a difetti software ignorati o a cattiva gestione del progetto, includono l'esplosione del razzo Ariane 5 e le letali overdosi di radiazioni somministrate dalla macchina medica Therac-25.

1.2 Definizione e Cenni Storici

Per rispondere alla cosiddetta crisi del software”, durante una conferenza NATO del 1968 venne coniato il termine *Software Engineering*”. L'Ingegneria del Software è oggi una disciplina ingegneristica che raggruppa teorie, metodi, tecniche e strumenti per sviluppare software di qualità in modo professionale. L'idea fondante è considerare il software come un prodotto ingegneristico qualsiasi (al pari di un edificio o una nave), concentrandosi in modo rigoroso sulla qualità del “processo di sviluppo” per ottenere sistemi di grandi dimensioni, a basso costo e in tempi brevi.

1.3 Le Fasi del Processo di Sviluppo

Il processo di sviluppo è una sequenza di attività che trasforma i requisiti in un prodotto finito. Si articola in diverse macro-fasi:

1.3.1 Ingegneria dei Requisiti

Definisce cosa” il sistema deve fare (funzionalità, servizi e vincoli) senza specificare il come”. È una fase di raccolta critica: commettere errori qui pregiudica la riuscita dell’intero progetto.

1.3.2 Design

Stabilisce “come” il sistema opererà. Si divide in *High Level Design* (definizione dell’architettura e delle relazioni tra le macro-componenti) e *Low Level Design* (struttura interna delle singole componenti). Generalmente, questa fase sfrutta il linguaggio di modellazione UML come standard.

1.3.3 Implementazione/Codifica

La traduzione materiale del design in codice sorgente. Se il design è estremamente dettagliato, questa può diventare un’attività puramente meccanica o automatizzata, come avviene nel *Model Driven Development* (MDD).

1.3.4 Testing (Verifica o Collaudo)

Esecuzione del software per individuare malfunzionamenti (*failure*) e valutare se il comportamento del sistema soddisfa i requisiti e i bisogni dell’utente. È un’attività molto costosa che assorbe circa il 40-50% dei costi di sviluppo di un’azienda.

1.3.5 Delivery e Manutenzione

Consegna, installazione e formazione degli utenti. La fase di manutenzione (ed evoluzione) garantisce l’operatività nel tempo, rimediando agli errori o adattando il software a nuovi sistemi operativi. L’evoluzione di un sistema costa circa 3 volte in più rispetto al suo sviluppo iniziale.

1.4 Ruoli Professionali

L’ingegneria del software richiede figure tecniche specializzate: l’analista per i requisiti, il designer per l’architettura, il programmatore (*coder*) per l’implementazione, il tester per il collaudo e il formatore per il rilascio. A coordinare il tutto vi è il *Project Manager*, un ruolo organizzativo responsabile della valutazione, pianificazione, gestione del personale e rispetto di tempi e costi.

2 Modelli di Processo di Sviluppo Software

2.1 Il Concetto di Processo di Sviluppo

L'ingegneria del software tratta il software come un prodotto industriale manifatturiero, applicando un rigoroso "processo di sviluppo". Un processo è un insieme strutturato e organizzato di attività il cui scopo è garantire ordine, controllo e ripetibilità. La modellazione di questo processo è fondamentale per incrementare la produttività e innalzare la qualità finale del software. Prima dell'inizio vero e proprio dello sviluppo, si esegue generalmente uno studio di fattibilità per valutare la convenienza tecnica ed economica (stima dei costi e ROI) del progetto.

2.2 L'Approccio "Code and Fix"

Pur essendo storicamente il primo approccio utilizzato, non viene considerato un vero e proprio modello ingegneristico. Consiste semplicemente nello scrivere codice e correggerlo ripetutamente, omettendo totalmente le fasi di analisi dei requisiti e di progettazione architettonica. Questa metodologia prettamente artigianale è applicabile solo a progetti di piccolissime dimensioni e si rivela inefficace e disastrosa per lo sviluppo di sistemi di scala aziendale.

2.3 Il Modello a Cascata (Waterfall) e il V-Model

Introdotta concettualmente nel 1956 e formalizzata nel 1970 da Winston Royce, segna il passaggio formale alla produzione industriale del software. Le attività procedono in sequenza lineare attraverso fasi distinte: Analisi dei Requisiti, Design, Implementazione (Codifica), Verifica (Testing) e Manutenzione. Ogni fase deve concludersi producendo un artefatto documentale che diventa l'input vincolante della fase successiva.

2.3.1 Pregi

Infonde rigida disciplina, richiede un'attenta pianificazione iniziale e pone forte enfasi su fasi critiche come il design e i requisiti. È ideale per sistemi in cui i requisiti sono fin dall'inizio chiari e immutabili.

2.3.2 Difetti

La sua natura monolitica impedisce il feedback tra le fasi. I requisiti vengono "congelati" all'inizio del progetto, rendendo il sistema incapace di adattarsi all'evoluzione del mercato; c'è il forte rischio di consegnare dopo anni un software che soddisfa requisiti ormai obsoleti.

2.3.3 Il V-Model

Per compensare i limiti di verifica strutturale del modello a cascata, è stato definito il **V-Model**. Questa variante pone massima enfasi sul testing, illustrando graficamente come ogni fase di progettazione e analisi (lato sinistro della "V") sia direttamente correlata e verificata da una specifica fase di collaudo (lato destro della "V").

2.4 Modelli Evolutivi: Prototipazione, Iterativi e Incrementali

I modelli evolutivi nascono proprio per abbattere la rigidità del modello sequenziale, abbracciando il cambiamento dei requisiti durante tutto il ciclo di vita. Questi approcci sviluppano il software esponendo agli utenti versioni iniziali e raffinandole in successivi rilasci (release) per ottenere feedback sul campo.

- **Prototipazione:** Si sviluppa rapidamente una versione rudimentale o parziale del sistema (un prototipo spesso "usa e getta") con l'obiettivo specifico di chiarire requisiti ancora ambigui o dubbi prima di procedere allo sviluppo reale.
- **Modello Incrementale:** Il prodotto viene suddiviso in sottomoduli. Ogni release consegna all'utente nuove funzionalità aggiuntive che si sommano a quelle già presenti.
- **Modello Iterativo:** Il sistema possiede da subito la totalità delle macro-funzionalità, ma sviluppate in forma molto grezza. Le release successive non aggiungono nuove funzioni, ma raffinano e ottimizzano le prestazioni e l'interfaccia di quelle già presenti.
- **Vantaggi dell'approccio Evolutivo:** Tempi di risposta rapidi al mercato, individuazione tempestiva degli errori di progettazione e continuo adattamento ai reali bisogni dell'utente finale.

2.5 Il Modello a Spirale

Proposto da Barry Boehm nel 1988, è l'approccio d'elezione per i sistemi software di massima dimensione e complessità. È un modello *risk-driven* (guidato dal rischio): lo sviluppo si evolve attraverso cicli a spirale, dove ogni spira rappresenta un avanzamento del prodotto. Ogni ciclo è diviso rigorosamente in quattro fasi: Pianificazione degli obiettivi, Analisi e risoluzione dei Rischi, Ingegnerizzazione (sviluppo) e Valutazione da parte del cliente. Individuare e mitigare il rischio preventivamente è il cuore di questo modello, ma richiede competenze di altissimo livello nell'analisi gestionale.

2.6 The Unified Process (UP)

Creato nel 1996 dall'unificazione delle metodologie dei padri del linguaggio UML (Booch, Rumbaugh e Jacobson), rappresenta un processo di tipo *plan-driven* per lo sviluppo orientato agli oggetti. L'intero ciclo sfrutta la notazione UML ed è fortemente guidato dai Casi d'Uso (*Use Case*). Lo sviluppo procede per iterazioni attraverso quattro macrolivelli temporali: *Inception* (avvio e fattibilità), *Elaboration* (design dell'architettura), *Construction* (codifica e test) e *Transition* (messa in esercizio).

2.7 Componenti e Metodi Agili

2.7.1 Sviluppo basato sui componenti (CBSE)

Approccio incentrato sull'assemblaggio di componenti software già esistenti e testati sul mercato. Riduce drasticamente le tempistiche e i costi, ma richiede compromessi sui requisiti (ci si deve accontentare di ciò che offre il componente) e comporta sfide nell'integrazione architetturale.

2.7.2 Metodi Plan-driven vs. Agili

I metodi tradizionali (*plan-driven*, come UP o Waterfall) si basano su piani rigorosi, ampia documentazione e contratti rigidi, risultando adatti a sistemi *safety-critical* o di scala immensa. I **Metodi Agili** (es. Extreme Programming e Scrum), emersi alla fine degli anni '90 per l'Internet economy, abbandonano la logica industriale in favore dell'adattabilità estrema. Si distinguono per cicli brevissimi (2-4 settimane), progettazione minimale, riorganizzazione continua del codice (*refactoring*), assenza di documentazione formale in favore della conoscenza tacita e strettissima interazione fisica con il cliente (*customer on-site*).

3 Ingegneria dei Requisiti

3.1 Definizione e Importanza

Un requisito software descrive una specifica funzionalità che il sistema dovrà offrire o un vincolo a cui dovrà sottostare. L'ingegneria dei requisiti si concentra sul definire "cosa" il sistema farà, e non il "come" verrà implementato. Questa è la fase più critica dell'intero ciclo di sviluppo: l'incompletezza o l'errata gestione dei requisiti è la causa principale di fallimento dei progetti software, incidendo per oltre il 44% dei casi. Un difetto introdotto in questa fase e scoperto solo durante il testing costa circa 15 volte di più per essere corretto rispetto a un difetto risolto in fase di design.

3.2 Tipologie di Requisiti

I requisiti si dividono in due categorie principali:

- **Requisiti Funzionali:** Descrivono i servizi e le azioni specifiche che il sistema deve eseguire (es. "Il sistema deve permettere la consultazione del saldo").
- **Requisiti Non-Funzionali:** Definiscono i vincoli operativi, di performance e tecnologici. Includono tempi di risposta, linguaggi da utilizzare, memoria occupata o vincoli etici e legislativi.

I requisiti variano anche per livello di astrazione:

- **Requisiti Utente:** Scritti in linguaggio naturale, servono per comunicare con il cliente e fungono da contratto.
- **Requisiti di Sistema:** Più dettagliati e tecnici, sono scritti per gli sviluppatori e rappresentano il punto di partenza per la successiva fase di progettazione.

3.3 Il Processo di Ingegneria dei Requisiti

Il processo è iterativo e si compone di diverse macro-attività:

- **Elicitazione (Cattura):** L'estrazione dei bisogni dagli *stakeholder* (chi ha un interesse nel sistema) tramite interviste, questionari, workshop o osservazione diretta. È una fase delicata soggetta a problemi di comunicazione, incomprensioni e assunzioni tacite; la stesura di un *Glossario* aziendale aiuta a mitigare l'ambiguità dei termini.
- **Analisi:** I bisogni raccolti vengono raffinati per valutarne la fattibilità. Si identificano i requisiti mancanti, si risolvono i conflitti (tramite negoziazione) e si stabilisce una priorità per decidere cosa è assolutamente necessario (MUST), desiderabile (SHOULD) o solo possibile (COULD).
- **Documentazione e Specifica:** I requisiti vengono formalizzati in un documento. Le notazioni spaziano dal linguaggio naturale testuale (spesso causa di ambiguità e verbosità), all'uso di *form* strutturati, fino a modelli visuali come gli Use Case o linguaggi matematici formali.
- **Validazione:** Verifica formale (spesso tramite *peer review* o prototipazione) per assicurarsi che i requisiti rispecchino i reali bisogni dell'utente e siano privi di difetti.
- **Gestione (Management):** L'attività continua di tracciamento e controllo dei cambiamenti ai requisiti e la valutazione dell'impatto che una modifica avrà sul resto del sistema.

3.4 Proprietà di un Buon Requisito

Un documento dei requisiti ben redatto deve rispettare precise proprietà:

- **Validità e Realismo:** Soddisfa realmente i bisogni ed è tecnologicamente realizzabile.
- **Completezza e Consistenza:** Non trascurare funzioni necessarie e non contiene requisiti in contraddizione tra loro.
- **Inequivocabilità:** Non deve prestarsi a interpretazioni multiple.
- **Verificabilità:** Deve essere espresso in modo misurabile per poter essere testato (es. sostituire "immediatamente" con "entro 3 secondi").
- **Tracciabilità:** Deve essere possibile collegare ogni riga di codice o test case al requisito originario, spesso avvalendosi di una Matrice di Tracciabilità.

4 Definizione dei Requisiti Basata su Use Case

4.1 Il Concetto di Use Case (Caso d'Uso)

Proposti nel 1992, i casi d'uso sono una tecnica per esprimere i requisiti funzionali di un sistema assumendo rigorosamente il punto di vista dell'utente finale. A differenza del classico requisito funzionale (che descrive cosa fa il sistema dal punto di vista della macchina), il caso d'uso racconta una "storia" di utilizzo, trattando il sistema come una scatola nera (*black-box*) con cui entità esterne interagiscono scambiandosi messaggi. Nonostante i diagrammi ad essi associati siano visuali (Use Case Diagram in UML), i casi d'uso veri e propri sono specificati interamente tramite testo strutturato.

4.2 Gli Elementi Fondamentali

La modellazione basata su Use Case si articola in quattro componenti principali:

- **Confini del Sistema:** La prima e più critica attività progettuale è tracciare il limite netto tra ciò che è parte del sistema da sviluppare e ciò che è esterno (sistemi preesistenti, ambiente).
- **Attori:** Rappresentano i ruoli assunti da entità esterne (persone, sensori hardware o altri sistemi) durante l'interazione. Un attore è un ruolo, non un individuo specifico (es. la stessa persona può essere "Cliente" e "Amministratore"). Si dividono in *Primari* (hanno un obiettivo attivo da raggiungere tramite il sistema) e *Secondari* (offrono un servizio al sistema o subiscono la sua azione). Gli attori possono essere organizzati in gerarchie tramite relazioni di generalizzazione/specializzazione.
- **Use Case:** Le azioni che gli attori possono compiere con il sistema.
- **Relazioni:** Le linee che indicano la partecipazione e l'interazione tra attori e casi d'uso.

4.3 Scenari e Template di Descrizione

Uno scenario è una singola e specifica sequenza temporale di messaggi/interazioni tra l'attore e il sistema (un singolo "cammino"). Un caso d'uso raggruppa sotto di sé l'insieme di tutti i possibili scenari accomunati dallo stesso scopo. La descrizione di un caso d'uso si appoggia a un *Template* (modello prestampato) comprendente:

- **Nome e Identificatore:** Il nome è il *goal* (l'obiettivo), espresso con una frase verbale attiva (es. PrelevaContante).
- **Attori Primari e Secondari.**
- **Precondizioni e Postcondizioni:** Lo stato del sistema richiesto prima dell'avvio e garantito al suo termine.
- **Scenario Principale (Flusso Base):** La sequenza di passi ideale, in un "mondo perfetto" senza errori.
- **Scenari Alternativi (Flussi Alternativi):** Le deviazioni, che indicano come gestire condizioni di errore o scelte secondarie.

Per descrivere logiche interne ai flussi si utilizzano costrutti di *Deviazione* (strutture logiche "Se/Altrimenti" per i bivi) e *Ripetizione* ("Per ogni" o "Fintantoché" per le operazioni cicliche).

4.4 Relazioni tra Casi d'Uso

I casi d'uso possono essere messi in relazione tra loro per favorire il riuso e la chiarezza del modello:

- **Inclusione (⊆):** Funziona come una chiamata a procedura/macro. Lo use case principale delega una parte del comportamento a uno use case incluso, essenziale per il completamento del primo. Serve per fattorizzare comportamenti comuni a più use case.
- **Estensione (⊇):** Specifica un comportamento opzionale (es. "Emissione multa" rispetto a "Restituisci libro scaduto"). Il caso base è completo in sé, ma definisce un "Punto di Estensione" (Extension point) dove il comportamento del caso esteso può innestarsi.
- **Generalizzazione/Specializzazione:** Un caso d'uso "figlio" (più specifico) eredita i passi del "padre", potendoli modificare (*override*) o aggiungendone di nuovi.

4.5 Linee Guida per un Buon Modello

I casi d'uso devono rimanere di alto livello, separando sempre il "cosa" dal "come". È un errore inserire al loro interno logiche di design o riferimenti stretti all'interfaccia grafica (es. vietato scrivere "L'utente preme il bottone

OK”, meglio ”L’utente conferma”). Bisogna inoltre evitare la trappola della scomposizione funzionale spinta. Sebbene siano eccellenti per i sistemi informativi, si rivelano poco efficaci per modellare sistemi guidati da requisiti puramente non-funzionali, sistemi *embedded* senza interfaccia utente o complessi processi algoritmici (es. elaborazioni matematiche silenti).

5 Design Architetturale

5.1 Il Concetto di Software Design

Il design è il processo fondamentale che trasforma un problema (definito dalla fase dei requisiti) in una soluzione strutturale, stabilendo ”come” il sistema dovrà operare. Questo processo si articola su due livelli di astrazione:

- **Architectural Design (High-level design):** Mappa i requisiti sull’architettura generale, individuando le macro-componenti e i sottosistemi.
- **Component Design (Low-level design):** Specifica in modo dettagliato la struttura interna delle singole componenti.

Contrariamente a quanto si pensi, il design non è un compito puramente creativo, ma un lavoro prevalentemente di routine basato sul riuso di soluzioni consolidate per problemi noti. Questo riuso può avvenire a vari livelli: tramite la clonazione di codice/design, l’applicazione di *Design Pattern* o l’adozione di *Stili Architetturali*.

5.2 Architettura Software

L’architettura software fornisce una visione d’insieme della struttura del sistema. Viene tipicamente rappresentata con diagrammi a blocchi (in UML si usano i *Component Diagram*) in cui i blocchi sono le componenti e le linee sono i connettori (es. invocazioni di metodi, pipe, eventi) che stabiliscono come queste componenti comunicano tra loro. Definire un’architettura offre enormi vantaggi: guida e semplifica lo sviluppo, supporta le decisioni gestionali (come il refactoring), favorisce il riuso su larga scala e costituisce la documentazione principale per comprendere le dipendenze del sistema.

5.3 Attributi di Qualità e Trade-off

Le scelte architetturali impattano direttamente sugli attributi qualitativi del prodotto finale:

- **Performance (Efficienza):** Favorita da componenti di grandi dimensioni (*large-grain*) per ridurre al minimo le comunicazioni esterne.
- **Security (Sicurezza dei dati):** Migliorata isolando le risorse critiche nei livelli più interni di un’architettura stratificata.

- **Safety (Sicurezza operativa):** Ottenuta confinando le funzioni *safety-critical* in pochi sottosistemi circoscritti.
- **Availability (Disponibilità):** Garantita inserendo componenti ridondanti e logiche di tolleranza ai guasti.
- **Maintainability (Manutenibilità):** Agevolata dall'uso di componenti molto piccole e specializzate (*fine-grain*), facilmente rimpiazzabili.

Questi attributi generano spesso dei **trade-off** (la cosiddetta "coperta corta"): ottimizzare la manutenibilità peggiora le performance per via del maggior numero di comunicazioni, così come massimizzare la sicurezza stratificata può degradare i tempi di risposta.

5.4 Stili Architetturali

Uno stile architetturale è un modello generico e riutilizzabile che definisce precise tipologie di componenti e connettori. I più diffusi includono:

- **Client-Server (Two-tier):** Divide il sistema tra chi richiede un servizio (Client) e chi lo fornisce (Server / Database). È altamente scalabile e favorisce il riuso, ma eredita le vulnerabilità dei sistemi distribuiti e un potenziale calo di efficienza nello scambio dati.
- **Call-Return Model:** Stile per sistemi sequenziali di tipo top-down. Un programma principale (*Driver*) possiede il controllo assoluto e chiama in sequenza le varie *routine* sottostanti.
- **Manager Model:** Ideale per sistemi concorrenti o *real-time* (come il controllo di un robot o l'allarme di una banca). Una componente centrale (*Manager*) monitora continuamente i sensori e orchestra in parallelo l'esecuzione di vari processi attuatori.
- **Microservices:** Evoluzione delle architetture a servizi (SOA). Un'applicazione non viene più costruita come un blocco "monolitico", ma come una suite di piccoli servizi indipendenti che comunicano tramite meccanismi leggeri (es. API REST). Questo risolve i problemi storici legati alla complessità, al deployment e all'isolamento dei bug.

Nella pratica ingegneristica, quasi tutti i sistemi di grandi dimensioni sono **eterogenei**, ovvero non seguono un singolo stile puro, ma mescolano più stili architetturali in modo gerarchico o combinato (come un compilatore che unisce l'architettura *pipe and filter* all'uso di un *repository* centrale per i dati).

6 Design delle Componenti (Low Level Design): Fasi e Principi

6.1 Fasi del Processo di Design

La progettazione del software non è un processo monolitico, ma si articola in una serie di attività a cascata e per raffinamenti successivi. Il processo parte dall'alto verso il basso:

- **Design dell'architettura:** Definisce le macro-componenti (sottosistemi) e i connettori che ne regolano la comunicazione. Questo livello è comunemente noto come *High Level Design*.
- **Specifica astratta e Design dell'interfaccia:** Si definiscono i componenti ad alto livello e si specificano le loro interfacce pubbliche.
- **Design delle componenti:** Corrisponde al *Low Level Design*, dove si entra nel dettaglio della singola classe Object-Oriented (OO) o del singolo modulo.
- **Design delle strutture dati e degli algoritmi:** Scelta delle strutture per memorizzare i dati e definizione logica (spesso tramite pseudocodice) delle procedure per manipolarli.

6.2 Design by Contract (Progettazione per Contratto)

Introdotta da Bertrand Meyer, il *Design by Contract* è un metodo di progettazione formale (fase di Low Level Design) che usa la metafora di un contratto legale tra entità software per incrementare la qualità del codice. Prevede la definizione di tre elementi fondamentali per ogni operazione, prima ancora della sua codifica:

- **Pre-condizione (Richiesta):** Un'espressione booleana che stabilisce le aspettative sullo stato del sistema *prima* che l'operazione venga chiamata. Ad esempio, per calcolare la radice quadrata di un numero, la pre-condizione è che il numero sia maggiore o uguale a zero.
- **Post-condizione (Garanzia):** Un'espressione booleana che descrive lo stato del sistema garantito *dopo* l'esecuzione dell'operazione, a patto che la pre-condizione fosse vera.
- **Invariante di classe:** Una condizione che tutti gli oggetti di quella classe devono rispettare quando sono in uno stato di "equilibrio" (ovvero prima e dopo ogni operazione, ma non necessariamente durante un calcolo intermedio).

Il contratto chiarisce nettamente le responsabilità, evitando la duplicazione inutile di controlli ("defensive programming"). Il **Cliente** (chi invoca l'operazione) ha il dovere di soddisfare la pre-condizione. Se il cliente lo fa, il **Fornitore** (chi

esegue l'operazione) ha il dovere di garantire la post-condizione e l'invariante. Se un'operazione fallisce per una pre-condizione non rispettata, la colpa è del cliente; se fallisce la post-condizione, la colpa è del fornitore. I vantaggi includono codice più pulito, miglior documentazione, facilità di testing e debugging.

6.3 Principi di Buona Progettazione

Progettare non è considerato un'attività puramente creativa, ma un lavoro ingegneristico che applica principi consolidati per ottenere software manutenibile, testabile e riusabile. Tra i principi cardine si trovano:

- **Astrazione:** Permette di descrivere il comportamento di un modulo omettendo i dettagli implementativi (Information Hiding). Riduce la complessità consentendo al progettista di concentrarsi solo sulle informazioni rilevanti a un certo livello.
- **Modularità e Accoppiamento:** Il software viene diviso in "moduli" (es. classi, package, funzioni) che forniscono servizi. I moduli comunicano tra loro creando delle dipendenze (accoppiamento tramite chiamate di funzioni, uso di tipi, ecc.). Un buon design minimizza l'accoppiamento: ad esempio, un modulo che dipende da un numero eccessivo di altri moduli (alto *fan-out*) è sintomo di cattiva progettazione e andrebbe scomposto.
- **Semplicità (KISS - Keep It Simple, Stupid):** Il design deve basarsi sul Rasoio di Ockham ("Non moltiplicare gli elementi più del necessario"). Tutto, dalle strutture dati alle interfacce, deve essere mantenuto il più semplice possibile.

7 Introduzione a UML 2.0

7.1 Definizione e Scopo

L'Unified Modeling Language (UML) è una famiglia di notazioni grafiche utilizzata per supportare la descrizione e la progettazione dei sistemi software, in particolare quelli Object-Oriented (OO). Si tratta di uno standard internazionale gestito dall'Object Management Group (OMG). Storicamente, UML nasce negli anni '90 dall'unificazione delle metodologie di Booch, Rumbaugh e Jacobson (noti come i "tre amigos"). È fondamentale chiarire cosa UML *non* è: non è un metodo di sviluppo (come il Waterfall o l'Agile), non è un linguaggio di programmazione tradizionale e non è vincolato a un processo specifico (come lo Unified Process), potendo essere integrato in qualsiasi contesto metodologico. UML fornisce una "lingua franca" universale per analisti e progettisti software.

7.2 Prospettive di Modellazione

Il significato degli elementi in un diagramma UML cambia radicalmente a seconda della prospettiva adottata:

- **Prospettiva Concettuale:** Si concentra sull'analisi del dominio aziendale (il "pezzo di mondo" in esame). Una "Classe", in quest'ottica, rappresenta un concetto del mondo reale (es. l'utente inteso come persona fisica). L'applicazione primaria di questa prospettiva è il **Modello di Dominio**, ovvero un glossario visuale delle astrazioni significative che aiuta la comunicazione tra stakeholder e ispira il successivo design del software.
- **Prospettiva Software:** Si concentra sulla progettazione tecnica. Qui, una "Classe" rappresenta un costrutto del linguaggio di programmazione (es. una classe Java con specifici attributi e metodi). Serve come reale documentazione e blueprint per gli sviluppatori.

7.3 Le Modalità di Utilizzo

L'UML può essere impiegato con diversi gradi di formalità:

- **Abbozzo (Sketch):** Diagrammi informali e spesso incompleti, tracciati su lavagne, con il solo scopo di facilitare la comunicazione e la discussione tra i membri del team.
- **Progetto Dettagliato (Blueprint):** Modelli completi e precisi creati con tool appositi, pensati per guidare un'implementazione meccanica da parte dei programmatori. Nella pratica industriale, tuttavia, questo approccio è considerato spesso troppo complesso, rigido e lento da realizzare.
- **Linguaggio di Programmazione:** Modelli eseguibili da cui specifici strumenti (es. WebRatio) generano automaticamente il codice sorgente dell'applicazione.

7.4 Viste, Diagrammi e il Meta-Modello

UML si fonda su una *Notazione* (la sintassi grafica) e su un *Meta-modello* (la definizione formale dei concetti stessi del linguaggio, come Classe o Attributo). Per descrivere sistemi complessi, si utilizza l'approccio delle **viste** (come il noto Modello 4+1 di Kruchten), che separano l'architettura in base all'osservatore: la vista logica per gli analisti, la vista di implementazione per i programmatori, la vista di *deployment* per i sistemisti, eccetera. Un singolo diagramma UML illustra un aspetto specifico di una di queste viste. Per coprire tutte queste necessità, la versione UML 2.0/2.5 definisce 14 tipi di diagrammi, categorizzati principalmente in Diagrammi Strutturali (es. Classi, Componenti) e Diagrammi Comportamentali (es. Casi d'Uso, Sequenza, Macchine a Stati). Pragmaticamente, la conoscenza del 20% di questi diagrammi è sufficiente per coprire l'80 lavoro ingegneristico standard.

7.5 Estensibilità: I Profili UML

Poiché l'UML generale potrebbe non bastare per modellare specifiche nicchie tecnologiche (come applicazioni Web o sistemi *embedded*), il linguaggio offre un meccanismo di estensione basato sui **Profili**. Un profilo arricchisce l'UML di base definendo:

- **Stereotipi:** Nuovi elementi derivati da quelli standard (spesso rappresentati con nuove icone o racchiusi tra << >>).
- **Vincoli (Constraint):** Regole aggiuntive che limitano l'uso di certi elementi per adattarli al nuovo contesto.

7.6 "Legalità" e Regole d'Uso

Non esiste una corrispondenza esatta 1:1 tra un diagramma UML astratto e il codice sorgente finale. Inoltre, data l'enorme complessità delle specifiche ufficiali OMG, i progettisti tendono a usare un "UML convenzionale" che piega alcune regole dello standard per favorire la leggibilità. La regola fondamentale raccomanda che la chiarezza comunicativa abbia sempre la precedenza sul rigore formale, e che omettere informazioni irrilevanti (la completezza è nemica della chiarezza) sia una pratica corretta e raccomandata.

8 Diagrammi delle Classi (UML 2.0)

8.1 Scopo e Prospettive

Il Class Diagram definisce la parte statica di un sistema, illustrando le classi, le loro caratteristiche (attributi e operazioni) e le relazioni che intercorrono tra esse (associazioni, aggregazioni, specializzazioni e dipendenze). Il significato del diagramma cambia radicalmente in base alla prospettiva adottata:

- **Prospettiva Concettuale:** Utilizzata in fase di analisi per modellare concetti del dominio ("il pezzo di mondo" di interesse). Una classe rappresenta un concetto reale e un'operazione una sua responsabilità.
- **Prospettiva Software:** Utilizzata in fase di design per modellare i moduli software. Una classe corrisponde a un vero e proprio costrutto del linguaggio orientato agli oggetti e un'operazione è destinata a essere implementata da un metodo.

8.2 Classi, Oggetti e Datatype

- **Classe:** Incapsula le caratteristiche comuni a un gruppo e funge da "stampo" per creare istanze. Si rappresenta graficamente con un rettangolo diviso in scomparti per il nome, gli attributi e le operazioni.

- **Oggetto:** È l'istanza di una classe. Possiede uno stato (definito dai valori degli attributi in un dato momento) e un comportamento (definito dalle operazioni).
- **Datatype:** A differenza degli oggetti, che possiedono una propria identità univoca, i datatype rappresentano puri valori intercambiabili (es. una Data, un importo in Euro, o enumerazioni come i Colori). Si indicano con lo stereotipo `«i»` o `«j»`.

8.3 Attributi e Operazioni

- **Attributi:** Definiscono lo stato. La sintassi formale è **visibilità nome: tipo [molteplicità] = default proprietà**. La visibilità può essere pubblica (+), privata (-), protetta (#) o di package (~). La molteplicità indica la quantità (es. 1, 0..1, per numero imprecisato, 1.. per almeno uno); se maggiore di 1, l'attributo modella un insieme di elementi.
- **Operazioni:** Definiscono il comportamento. La sintassi è **visibilità nome (parametri): tipo-ritornato**. I parametri includono una direzione (in per l'input, out per l'output). Le operazioni si dividono in *query* (restituiscono un valore senza alterare lo stato) e *modificatori* (alterano lo stato dell'oggetto). Bisogna distinguere concettualmente l'operazione (la dichiarazione astratta) dal metodo (il corpo del codice che la implementa fisicamente).
- **Elementi Statici:** Attributi e operazioni condivisi a livello di classe (non legati a una singola istanza, come i costruttori o le costanti matematiche) vengono rappresentati graficamente con una sottolineatura.

8.4 Relazioni e Associazioni

- **Associazioni:** Rappresentano connessioni fisiche o concettuali tra classi. Possono indicare un verso di navigazione (una freccia aperta per mostrare in quale direzione è possibile reperire le informazioni) o esplicitare i ruoli assunti dalle classi agli estremi.
- **Molteplicità delle Associazioni:** Indica il numero di istanze (link) ammesse tra gli oggetti connessi. Ad esempio, "un dipartimento è situato in 1..* sedi".
- **Associazioni Bidirezionali:** Navigabili in entrambe le direzioni. A livello software sono complesse da implementare perché richiedono logiche di sincronizzazione per mantenere la coerenza dei dati (es. se "Mario" sposa "Anna", bisogna aggiornare simultaneamente sia l'oggetto Mario che l'oggetto Anna).
- **Associazioni Riflessive:** Si verificano quando una classe ha un'associazione con se stessa (es. una Cartella informatica che contiene altre Cartelle).

- **Classi Associative:** Vengono impiegate quando alcune proprietà appartengono alla relazione stessa e non alle singole classi coinvolte (es. il "voto" di un esame appartiene all'associazione tra uno Studente e un Corso, non al solo studente o al solo corso). A livello pratico, spesso si preferisce trasformarle in classi normali per semplicità.

8.5 Diagrammi degli Oggetti (Object Diagram)

I diagrammi degli oggetti vengono utilizzati per chiarire diagrammi delle classi troppo complessi o astratti. Mostrano le istanze reali (rappresentate come nomeOggetto: NomeClasse, rigorosamente sottolineate) e i collegamenti (*link*), esemplificando come il sistema si comporterebbe in uno scenario pratico valido.

9 Interfacce e Vincoli nei Diagrammi delle Classi (UML 2.0)

9.1 Il Concetto di Interfaccia

Nella progettazione orientata agli oggetti, il termine "interfaccia" viene utilizzato principalmente in due accezioni: come l'insieme delle operazioni visibili all'esterno di un oggetto, oppure come un'entità strutturale simile a una classe, ma priva di implementazione. Un'interfaccia contiene, di fatto, solo le dichiarazioni delle operazioni pubbliche. Nel design di un sistema, una o più classi possono *fornire* (cioè implementare) un'interfaccia, mentre altre classi possono *richiederla* per poter funzionare correttamente. L'utilizzo delle interfacce è una pratica fondamentale per separare l'implementazione effettiva di una classe dalla sua forma esteriore: questo approccio diminuisce drasticamente l'accoppiamento tra le classi, permettendo di variare l'implementazione interna senza dover modificare il codice dei moduli clienti.

9.2 Rappresentazione delle Interfacce in UML

Esistono due modi principali per rappresentare un'interfaccia e il suo utilizzo all'interno di un Class Diagram:

- **Stereotipo `«interface»`:** L'interfaccia viene disegnata come un normale blocco di classe, ma contrassegnata dallo stereotipo `<>`. La relazione tra la classe che implementa l'interfaccia e l'interfaccia stessa è indicata da una freccia tratteggiata con punta a triangolo vuoto (relazione di realizzazione).
- **Notazione "Lollipop":** È una notazione visiva più compatta. L'interfaccia *fornita* (esposta) da una classe viene disegnata come un segmento che termina con un cerchio (la "pallina" o lollipop). L'interfaccia *richiesta* da un'altra classe viene invece rappresentata con un semicerchio (il "socket") che avvolge la pallina, indicando visivamente l'incastro e la dipendenza tra fornitore e cliente.

9.3 Regole di Vincolo e OCL

Un diagramma delle classi definisce intrinsecamente dei vincoli strutturali attraverso le molteplicità, le associazioni e le generalizzazioni. Tuttavia, questi costrutti grafici di base non sono sufficienti per esprimere regole di business più complesse e specifiche. Per ovviare a questo limite e specificare vincoli aggiuntivi, UML si appoggia a espressioni testuali che possono essere scritte in linguaggio naturale, in pseudocodice, oppure, per garantire il massimo rigore matematico, tramite l'**OCL (Object Constraint Language)**. L'OCL è un linguaggio formale standardizzato e integrato in UML, basato sulla logica del primo ordine, che permette di definire in modo inequivocabile vincoli, pre-condizioni e post-condizioni direttamente sul modello.

10 Diagrammi di Sequenza e di Comunicazione (UML 2.0)

10.1 Diagrammi di Interazione

Nella modellazione dinamica di un sistema, i diagrammi di interazione descrivono la collaborazione tra un gruppo di oggetti, ovvero lo scambio di messaggi necessario per raggiungere un determinato obiettivo. I messaggi scambiati tra le entità possono essere:

- **Sincroni:** Il mittente invia la richiesta e si pone in attesa del risultato.
- **Asincroni:** Il mittente invia il messaggio e continua immediatamente la propria esecuzione (tipicamente impiegati per rappresentare processi concorrenti).

10.2 Notazione del Diagramma di Sequenza

I diagrammi di sequenza (Sequence Diagram) si sviluppano su due dimensioni: l'asse orizzontale mostra i partecipanti, mentre l'asse verticale rappresenta il tempo che scorre verso il basso. Gli elementi fondamentali includono:

- **Partecipanti e Linee di Vita:** Ogni partecipante è rappresentato in alto con il suo nome (e/o tipo) e possiede una "linea di vita" tratteggiata verticale che indica la sua esistenza nel tempo.
- **Barra di Attivazione (Specifica di esecuzione):** Un rettangolo sovrapposto alla linea di vita che evidenzia l'intervallo temporale in cui un oggetto ha il controllo o sta eseguendo un'operazione.
- **Creazione e Distruzione:** La creazione dinamica di un oggetto viene rappresentata indirizzando un messaggio (es. **new**) direttamente al blocco dell'oggetto appena nato. La distruzione esplicita di un'entità è segnalata con una **X** al termine della sua linea di vita, utile per sistemi dotati di garbage collector per indicare che la memoria può essere liberata.

- **Tipi di chiamate:** Oltre alle normali comunicazioni tra oggetti diversi, è possibile modellare messaggi di ritorno e messaggi *Self*, ovvero chiamate interne a metodi appartenenti alla classe stessa.

10.3 Logica di Controllo e Frame di Interazione

Sebbene i Sequence Diagram siano ideali per mostrare le interazioni, non sono adatti per esprimere logiche algoritmiche complesse. Per sopperire a questo, l'UML 2.0 utilizza i "Frame", cornici che permettono di raggruppare blocchi di messaggi per modellarne il flusso di controllo tramite specifici operatori:

- **loop:** Esegue un ciclo fintantoché una guardia booleana risulta vera.
- **opt:** Esegue il frammento in modo opzionale, solo se la condizione è verificata.
- **alt:** Modella frammenti alternativi (corrispondente al costrutto *If-Then-Else* o *Switch/Case*).
- **par:** Esegue i blocchi contenuti rigorosamente in parallelo.
- **ref:** Permette di richiamare un altro diagramma di sequenza, funzionando in modo analogo alla chiamata di una procedura esterna, alleggerendo la complessità visiva.

Nonostante questi strumenti, per descrivere l'algoritmica di dettaglio è considerata best practice utilizzare lo pseudocodice piuttosto che i Frame, per evitare di appesantire eccessivamente la notazione visiva.

10.4 Utilizzo nei Requisiti e nel Design

L'uso dei Sequence Diagram varia drasticamente a seconda del livello di astrazione:

- **Livello di Requisiti (System Sequence Diagram - SSD):** Illustrano gli eventi di input e output tra gli Attori esterni e il "Sistema" considerato come una singola entità scatola nera (*black-box*). Sono fondamentali per stabilire con precisione i confini e le operazioni del sistema, e fungono da base cruciale per i successivi casi di test.
- **Livello di Design:** Entrano nel dettaglio della *Business Logic* e mostrano come i messaggi fluiscono concretamente da un oggetto software all'altro (le reali chiamate ai metodi) per realizzare un determinato scenario operativo.

10.5 Diagrammi di Comunicazione

I diagrammi di comunicazione (precedentemente noti come diagrammi di collaborazione) forniscono esattamente le stesse informazioni semantiche dei diagrammi di sequenza, ma presentate con una prospettiva visiva rovesciata. Mentre i Sequence Diagram pongono massima enfasi sull'ordine temporale cronologico, i diagrammi di comunicazione si concentrano sulle relazioni topologiche e sui collegamenti diretti tra gli oggetti (i *link*), affidando l'ordine cronologico a una numerazione progressiva posta sui messaggi.

11 Diagrammi di Macchine a Stati (State Machine Diagram)

11.1 Definizione e Scopo

I diagrammi di macchine a stati descrivono il comportamento dinamico di un'entità (che sia un sistema, una classe software o un dispositivo hardware) come variazione del suo stato interno in risposta a sollecitazioni esterne. A differenza dei diagrammi di interazione, che si concentrano sullo scambio di messaggi tra più oggetti, la macchina a stati descrive la sequenza di stati che un singolo oggetto attraversa durante il suo ciclo di vita.

11.2 Il Concetto di Stato

Uno stato rappresenta una situazione specifica nella vita di un oggetto durante la quale determinate condizioni sono soddisfatte e specifiche attività possono essere eseguite. Non va confuso con la semplice combinazione dei valori delle variabili in un dato istante: a livello di modellazione UML, uno stato è un'astrazione più alta che indica un momento in cui il sistema si comporterà in modo differente al verificarsi degli stessi eventi.

11.3 Elementi Grafici e Sintassi

Le macchine a stati sono rappresentate come grafi diretti:

- **Nodi (Stati):** Rappresentano gli stati in cui l'entità può trovarsi. L'esecuzione parte sempre da un unico *Nodo Iniziale* (pseudo-stato indicato da un disco nero) e può concludersi in uno o più *Nodi Finali* (disco nero bordato).
- **Archi (Transizioni):** Indicano il passaggio da uno stato di origine a uno di destinazione. La sintassi di una transizione è **evento** [**guardia**] / **attività**.

Le transizioni sono guidate da tre elementi, tutti opzionali:

- **Evento:** Lo stimolo (ricezione di un messaggio, scatto di un timer o cambiamento di una condizione) che innesca il passaggio di stato.

- **Guardia:** Una condizione booleana che, solo se valutata come vera, permette alla transizione di avvenire.
- **Attività:** Una o più azioni atomiche eseguite durante il transito. Possono essere descritte in linguaggio naturale, pseudocodice o linguaggi formali come ALF.

11.4 Semantica di Esecuzione

La macchina a stati riceve gli eventi e li inserisce in una coda, elaborandoli uno alla volta in base al principio di *run-to-completion*: un evento viene processato solo quando il precedente ha terminato la sua esecuzione. Se, per un dato evento, esistono più transizioni valide contemporaneamente, il sistema ne esegue solo una compiendo una scelta non deterministica.

11.5 Attività Interne

Oltre alle attività scatenate durante le transizioni, uno stato può reagire a eventi eseguendo operazioni senza cambiare stato. Esistono specifiche attività interne legate al ciclo di vita dello stato stesso:

- **entry:** Eseguita in modo istantaneo appena l'oggetto entra nello stato.
- **exit:** Eseguita in modo istantaneo quando l'oggetto esce dallo stato.
- **do (do-activity):** Un'attività che viene eseguita per tutto il tempo in cui l'oggetto permane nello stato. A differenza delle altre, non è istantanea, dura nel tempo e può essere interrotta istantaneamente dal verificarsi di un nuovo evento esterno.

11.6 Stati Compositi e Concorrenza

Per gestire la complessità, gli stati possono essere "compositi", ovvero contenere al loro interno altri sotto-stati. Questo approccio permette di raggruppare visivamente macro-comportamenti. Gli stati compositi sono fondamentali anche per modellare la **concorrenza**: lo stato può essere diviso in regioni ortogonali, eseguendo sotto-diagrammi in modo rigorosamente parallelo (thread multipli). Per consentire l'ingresso e l'uscita fluida in punti specifici di uno stato composito molto articolato, si utilizzano gli *Entry point* e *Exit point*.

11.7 Applicazione e Implementazione Software

L'uso delle macchine a stati è raccomandato esclusivamente per entità dotate di una logica interna complessa e fortemente reattiva (es. sistemi di controllo integrati, distributori automatici, o cicli di vita documentali complessi). A livello di codice sorgente, l'implementazione basata su un grande costrutto **switch** è sconsigliata perché difficile da mantenere ed estendere. La soluzione ingegneristica d'elezione è il **Design Pattern State**, che incapsula i comportamenti

di ogni singolo stato all'interno di classi separate derivanti da una gerarchia comune, delegando la logica al polimorfismo. In alternativa, è prassi comune mappare il diagramma su una Tabella di Stato per poi generare il codice in modo automatico tramite tool dedicati (come SMC).

12 Diagrammi delle Attività (Activity Diagram)

12.1 Definizione e Contesti d'Uso

I diagrammi delle attività descrivono come viene svolta un'attività, illustrando il flusso di azioni, la loro sequenza e la loro semantica. A differenza dei tradizionali diagrammi di flusso (flowchart), questi diagrammi supportano nativamente l'elaborazione concorrente e parallela. Sono strumenti estremamente poliedrici utilizzati in diverse fasi dello sviluppo: in fase di analisi per modellare processi di business o flussi dei casi d'uso, in progettazione per definire algoritmi o singole operazioni di una classe (come alternativa allo pseudocodice), e persino in codifica come UML eseguibile.

12.2 La Semantica dei Token

L'esecuzione di un diagramma delle attività si basa sul concetto di *token*, entità simili a pedine che viaggiano lungo i flussi del diagramma. Un nodo d'azione viene eseguito solo quando sono presenti token su tutti i suoi archi in entrata. Al termine della sua esecuzione, l'azione genera nuovi token su tutti gli archi in uscita, permettendo al flusso di procedere attraverso il diagramma.

12.3 Elementi Fondamentali: Nodi e Flussi

Il diagramma è composto da flussi (freccie) che collegano tre categorie principali di nodi:

- **Nodi Azione:** Specificano l'unità elementare di comportamento. Un'azione può essere descritta in modo informale in linguaggio naturale, scritta in pseudocodice, o utilizzata per invocare un'operazione di classe o un'altra intera attività. Il simbolo del "Rake" (un piccolo tridente) posto all'interno del nodo indica proprio che quell'azione non è elementare, ma è esplosa in dettaglio in un diagramma di attività separato.
- **Nodi Oggetto:** Rappresentano gli oggetti (concettuali o software) utilizzati come input o output dalle azioni. È possibile specificare lo stato di un oggetto racchiudendolo tra parentesi quadre (es. `[finito]`), in stretta coerenza con l'eventuale macchina a stati associata a quell'oggetto. I nodi oggetto posizionati fisicamente sul bordo del diagramma fungono da parametri di input e valori di ritorno per l'intera attività.
- **Nodi di Controllo:** Gestiscono e incanalano la logica del flusso.

12.4 Gestione del Flusso (Nodi di Controllo)

- **Nodi Iniziali e Finali:** Un disco nero marca l'inizio dell'attività e genera il primo token esecutivo. In ogni diagramma deve esserci un solo nodo iniziale. L'attività termina globalmente quando un token raggiunge il *nodo finale di attività* (un disco nero bordato), il quale distrugge tutti i token attivi nel diagramma. Esiste anche il *nodo finale di flusso* (un cerchio con una X al centro), che fa terminare esclusivamente il singolo cammino (thread) che lo ha raggiunto, senza interrompere gli altri flussi paralleli dell'attività.
- **Decisione e Fusione (Decision/Merge):** Entrambi rappresentati visivamente da rombi. Il nodo di *decisione* riceve un flusso in ingresso e lo indirizza su uno dei possibili output valutando delle condizioni booleane definite "guardie". Se più guardie risultano vere contemporaneamente, la scelta del percorso è non deterministica; se nessuna è vera, l'attività si blocca. Il nodo di *fusione* compie l'operazione opposta, raccogliendo percorsi logici alternativi per convogliarli in un singolo flusso in uscita.
- **Fork e Join:** Rappresentati da barre nere spesse. Il nodo *Fork* divide un flusso sequenziale in più flussi eseguiti parallelamente, duplicando i token in ingresso su tutte le uscite. Il nodo *Join* sincronizza questi processi paralleli: attende rigorosamente che arrivino i token su tutti i suoi ingressi prima di far procedere l'esecuzione, emettendo un singolo token unificato in uscita.

12.5 Eventi e Partizioni

- **Eventi e Regioni Interrompibili:** Specifici nodi d'azione gestiscono la comunicazione con l'esterno. L'invio di un evento ad un'altra entità è un'operazione asincrona e non blocca il flusso dell'attività. La ricezione di un evento (o l'attesa di un evento temporale basato su scadenze, rappresentato da una clessidra) blocca invece l'esecuzione finché la condizione o lo stimolo non si verifica. Per gestire eccezioni o situazioni anomale (es. la richiesta di annullamento di un ordine in corso), si utilizzano le *Regioni Interrompibili*: se un token si trova al loro interno nel momento in cui arriva il segnale di interruzione, le operazioni in quella regione vengono terminate forzatamente.
- **Partizioni (Swimlanes):** Organizzano il diagramma in corsie orizzontali o verticali per specificare chi esegue una determinata azione, chiarendo il partecipante o il ruolo responsabile. Queste suddivisioni non modificano minimamente la semantica di base del flusso, ma aggiungono un livello di leggibilità e informazione cruciale.

13 UML 2.0: Component, Package e Deployment Diagram

13.1 Diagrammi dei Componenti (Component Diagram)

I diagrammi dei componenti offrono una vista architeturale del sistema (*High Level Design*), mostrando le componenti software e le dipendenze tra di esse. Modellare l'architettura a livello di singole classi risulta inefficace per sistemi complessi (le classi sono a "grana troppo fine"); i componenti permettono invece di avere una panoramica chiara senza perdersi nei dettagli.

- **Concetto di Componente:** Un componente è un'unità modulare con interfacce ben definite, sostituibile e componibile all'interno del suo ambiente. Funge da "scatola nera" il cui comportamento esteriore è dettato unicamente dalle sue interfacce. A livello fisico, queste entità logiche sono poi realizzate tramite *artefatti* (come file JAR, servizi Web REST, ecc.).
- **Notazione:** Si rappresenta con un rettangolo contenente il nome e la parola chiave <> o una specifica icona.
- **Interfacce e Porte:** Le interfacce si dividono in *fornite* (ciò che il componente offre, notazione "lollipop" a pallina) e *richieste* (ciò che il componente necessita per funzionare, notazione "socket" a semicerchio). Un insieme di interfacce semanticamente coese può essere raggruppato in una **Porta**, che funge da punto specifico di interazione tra l'esterno e l'interno del componente.
- **Vista Interna:** È possibile specificare la struttura interna di un componente (mostrando i sub-componenti o le classi che lo realizzano), utile per chi dovrà materialmente scriverne il codice.

13.2 Diagrammi di Dislocamento (Deployment Diagram)

Si tratta di una vista fortemente orientata all'implementazione che illustra la relazione fisica tra l'hardware e il software all'interno del sistema.

- **Nodi e Connessioni:** Il diagramma è composto da *nodi*, che rappresentano risorse computazionali. Un nodo può essere un dispositivo hardware fisico (<>, es. un server o un PC) oppure un ambiente di esecuzione software (<>, es. un browser, una macchina virtuale o un container). Le *connessioni* tra i nodi indicano i canali di comunicazione (es. TCP/IP).
- **Artefatti:** Rappresentano le entità fisiche e concrete del mondo reale (es. file sorgente, eseguibili, script, tabelle di database, file `.war`) che vengono installate ("deployate") sui nodi.
- **Relazione `⚙`:** Crea un collegamento diretto tra il Deployment Diagram e il Component Diagram. Indica che un artefatto fisico (es. `GestioneOrdini.jar`)

è la manifestazione o realizzazione concreta di uno o più componenti logici definiti in fase di design.

13.3 Diagrammi dei Pacchetti (Package Diagram)

I package diagram sono essenziali per gestire la complessità strutturale del codice in sistemi di medie o grandi dimensioni. Un package è un costrutto che raggruppa un numero arbitrario di elementi UML (principalmente classi, ma anche altri package, creando gerarchie).

- **Namespace e Visibilità:** Ogni package definisce un *namespace* (uno spazio di nomi univoci). Per riferirsi a una classe specifica si utilizza il nome completamente qualificato (es. `System::Data`). Gli elementi al suo interno possono avere visibilità pubblica (+, visibili all'esterno) o privata (-, a uso interno).
- **Dipendenze:** Una dipendenza tra due package (es. *A* dipende da *B*) significa che almeno una classe nel package *A* dipende da una classe pubblica presente nel package *B*.
- **Principi di Progettazione:** La suddivisione in package deve seguire principi rigorosi come l'alta coesione, il basso accoppiamento e il *Common Reuse Principle* (le classi riusate assieme dovrebbero stare nello stesso package).
- **Gestione dei Cicli:** È fondamentale rispettare l'*Acyclic Dependency Principle*, evitando che le dipendenze tra package formino dei cicli chiusi, i quali causerebbero modifiche a cascata. Se si rileva un ciclo, questo va risolto tramite la *fusione* dei package coinvolti o la *separazione* (estrazione) delle entità problematiche in un nuovo package.

14 Software Design Patterns

14.1 Definizione e Scopo

Un Design Pattern è una soluzione elegante e consolidata a uno specifico problema ricorrente di progettazione o programmazione orientata agli oggetti (OO). Non si tratta della prima soluzione che viene in mente, ma di un'unità di riuso astratta e indipendente dal linguaggio di programmazione, formalizzata nel 1995 dal celebre libro della "Gang of Four" (GoF). L'uso dei pattern serve a:

- Applicare i principi di buona progettazione OO (es. alta coesione e basso accoppiamento).
- Facilitare la comunicazione tra sviluppatori fornendo un vocabolario comune e preciso.

14.2 Pattern, Librerie e Framework

È fondamentale distinguere i pattern da altre unità di riuso:

- **Libreria:** Lo sviluppatore scrive il codice principale (main) che chiama le operazioni fornite dalla libreria.
- **Framework:** Realizza un'inversione del flusso di controllo (il cosiddetto *Hollywood principle*: "Non chiamateci, vi chiameremo noi"). Il framework detta l'architettura generale e chiama il codice (le classi e le operazioni) che lo sviluppatore scrive per "riempire i buchi".
- **Design Pattern:** A differenza dei framework, i pattern sono più astratti, più piccoli e non sono legati a uno specifico dominio applicativo; vanno sempre implementati e adattati al contesto.

14.3 Pattern GRASP

I pattern GRASP (General Responsibility Assignment Software Patterns) sono pattern elementari che stabiliscono i principi per l'assegnazione delle responsabilità nel software, costituendo il fondamento per la progettazione di sistemi OO.

14.4 I Pattern Classici (GoF)

Il catalogo GoF originale include 23 pattern, descritti tramite un template preciso (Nome, Problema, Soluzione, Conseguenze). Si dividono in tre macro-categorie:

14.4.1 Pattern Creazionali

Gestiscono la creazione controllata degli oggetti.

- **Abstract Factory:** Fornisce un'interfaccia per creare famiglie di oggetti connessi tra loro, senza che il *client* debba specificare le loro classi concrete. Permette di cambiare facilmente l'intera famiglia di prodotti in uso e garantisce la consistenza, ma rende difficile l'aggiunta di nuove tipologie di prodotti, poiché richiede la modifica di tutte le fabbriche esistenti.

14.4.2 Pattern Strutturali

Si occupano della composizione di classi e oggetti in strutture più complesse.

- **Adapter:** Converte l'interfaccia di una classe in un'altra interfaccia attesa dal cliente, permettendo a classi incompatibili di collaborare.
- **Facade (Facciata):** Fornisce un'interfaccia unica e semplificata per accedere a un intero sottosistema complesso. A differenza dell'Adapter che *converte* un'interfaccia, la Facade la *semplifica*, promuovendo un accoppiamento debole tra cliente e sottosistema.

14.4.3 Pattern Comportamentali

Definiscono come le classi interagiscono e si distribuiscono le responsabilità.

- **Template Method:** Definisce lo scheletro di un algoritmo in un metodo di una superclasse, delegando l'implementazione di alcuni passi (le parti variabili) alle sottoclassi. Realizza un'inversione di controllo ed è ampiamente usato nei framework.
- **Observer:** Definisce una dipendenza "uno a molti" (tipo *broadcast* o *publish/subscribe*). Quando un oggetto (osservato) cambia stato, tutti i suoi dipendenti (osservatori) vengono notificati automaticamente. Promuove un accoppiamento debole, ma può causare aggiornamenti a cascata inaspettati.
- **State:** Permette a un oggetto di cambiare il proprio comportamento a *run-time* al variare del suo stato interno. Estrae la logica di ogni stato in classi separate organizzate in gerarchia, eliminando l'uso di complessi e rigidi costrutti `switch` e facilitando l'implementazione delle macchine a stati.

15 Code Refactoring

15.1 Definizione e Scopo

Il *Code Refactoring* è una modifica apportata alla struttura interna del software per renderlo più semplice da comprendere e meno costoso da modificare, senza alterarne in alcun modo il comportamento osservabile dall'esterno. Si tratta, in sintesi, di una riorganizzazione e ristrutturazione del codice, tipicamente orientato agli oggetti. È fondamentale chiarire cosa il refactoring *non* è: non è un'attività di correzione di bug (bug fixing), non consiste nel tradurre il codice in un altro linguaggio, non stravolge il design generale del sistema e non è una migrazione verso una nuova piattaforma. Gli obiettivi primari di questa pratica includono: limitare il degrado o l'erosione del design (design decay) che avviene naturalmente durante l'evoluzione del software, aumentare la leggibilità del codice, semplificare le future fasi di manutenzione e facilitare il testing. A lungo termine, un codice sottoposto a refactoring continuo mantiene bassi i costi e i tempi di modifica rispetto a un codice trascurato.

15.2 Quando Applicare il Refactoring

Il refactoring non deve essere un'attività isolata o pianificata massivamente (ad esempio, "due settimane di refactoring ogni due mesi"). Al contrario, deve essere applicato il più spesso possibile durante il normale ciclo di sviluppo. I momenti ideali per applicarlo sono:

- Quando si deve aggiungere una nuova funzionalità: rifattorizzare preventivamente aiuta a inserire la novità in modo più pulito e veloce.

- Quando si corregge un bug.
- Quando si rileva un cosiddetto "Code Smell".

15.3 I Code Smell (Le "Puzze" del Codice)

Un *Code Smell* è un indicatore o un sintomo che suggerisce un potenziale problema nella struttura del codice. Non dà la certezza matematica di un errore, ma segnala aree che potrebbero necessitare di un miglioramento progettuale. Spesso i tool di analisi delle metriche software (come STAN4J) evidenziano automaticamente questi smell (ad esempio, classi con scarsa coesione o metodi eccessivamente lunghi). I code smell si dividono in categorie principali:

- **"Troppo codice"**: Include metodi troppo lunghi (*Long method*), classi enormi che accentrano troppe responsabilità (*Large/God/Blob class*), liste di parametri interminabili e codice morto (*Dead code*). Un problema particolarmente grave è il **codice duplicato (Software Clones)**, presente dal 5% al 20% nei sistemi software, che causa la propagazione rapida dei bug e ostacola la manutenzione.
- **"Non abbastanza codice"**: Include classi con poco codice, clausole di *catch* vuote, o le cosiddette *Data Class* (classi che possiedono solo attributi e metodi getter/setter, le quali generano un accoppiamento elevato perché costringono le altre classi a manipolarne continuamente i dati).
- **Struttura relazionale**: Come le *Message Chains* (lunghe catene di chiamate di metodi come `a.getB().getC().doSomething()`), che denotano cattiva distribuzione delle responsabilità.

15.4 Il Processo e la Sinergia con il Testing

L'operazione di refactoring è intimamente legata al software testing. Il "ritmo" ingegneristico corretto prevede passaggi rigorosi e brevi:

1. Identificare il code smell.
2. Determinare e applicare una piccola modifica strutturale guidata da un catalogo standard di refactoring.
3. Compilare.
4. Eseguire immediatamente l'intera suite di test per assicurarsi di non aver "rotto" nulla (nessun cambiamento nel comportamento). Se il test fallisce, si annulla la modifica. È fondamentale non fare "il passo più lungo della gamba" cercando di cambiare troppe cose contemporaneamente.

15.5 Strumenti di Supporto

Il refactoring può essere svolto manualmente, ma la pratica industriale raccomanda l'uso di tool integrati per evitare errori a cascata (ad esempio, rinominare una classe manualmente richiede di aggiornare costruttori, file, e riferimenti in tutto il progetto). Gli IDE moderni (come Eclipse) incorporano refactoring semplici (Rename, Extract Interface, Pull-Up, Push-Down). Esistono poi plug-in avanzati (come JDeodorant per Eclipse o ReSharper per Visual Studio) in grado di identificare automaticamente gli smell più complessi e suggerire la strategia risolutiva. Tuttavia, anche utilizzando strumenti automatici, la ri-esecuzione dei casi di test rimane un obbligo imprescindibile.

15.6 Catalogo: Esempi di Refactoring Comuni

I refactoring di basso livello (low-level) costituiscono i mattoni per eseguire ristrutturazioni più complesse (come l'introduzione di un Design Pattern). Alcune tecniche classiche includono:

- **Extract Method:** Preleva un blocco di codice complesso (o duplicato) da un metodo e lo isola in un nuovo metodo privato, sostituendo il blocco originale con una chiamata a quest'ultimo.
- **Move Method:** Sposta un metodo verso la classe con cui collabora maggiormente. Si usa quando una classe ha troppo "behaviour" o le classi sono troppo accoppiate (risolve il problema delle *Message chains*).
- **Replace Temp with Query:** Sostituisce una variabile locale (che memorizza il risultato di un'espressione) con un metodo "query" dedicato. Rende l'informazione accessibile a tutta la classe e semplifica i metodi.
- **Replace Parameter with Method:** Se un metodo calcola un valore solo per passarlo come parametro a un altro metodo, è preferibile far sì che sia il metodo ricevente a invocare direttamente il calcolo, alleggerendo la firma della funzione.
- **Extract Class:** Risolve il problema delle *God Class*. Se una classe fa troppe cose, se ne crea una nuova spostandovi una parte logicamente coesa di attributi e operazioni.
- **Pull-Up / Push-Down:** Sposta attributi o metodi comuni dalle sotto-classi alla superclasse (*Pull-Up*) o viceversa (*Push-Down*) per massimizzare il riuso tramite ereditarietà.
- **Refactoring Complessi:** Includono *Replace Inheritance with Delegation* (sostituire l'ereditarietà con la composizione quando una sottoclasse usa solo una minima parte dei metodi della superclasse, es. derivare uno Stack da un Vector), *Replace Conditional with Polymorphism* (eliminare gli switch usando classi polimorfiche) e *Separate Domain from Presentation* (estrarre la logica di business intrappolata nel codice della User Interface).

Il principio guida del refactoring, come sottolineato da Martin Fowler, è che un sistema composto da tanti metodi piccoli è infinitamente più semplice da estendere, comprendere e mantenere rispetto a un sistema con pochi metodi monolitici.

16 Introduzione al Software Testing

16.1 Definizione e Complessità

Il software testing è una procedura sistematica che prevede l'esecuzione di un sistema software (SUT - System Under Test) con l'intento primario di trovare malfunzionamenti. Il testing esaustivo (testare ogni possibile combinazione di input) è matematicamente irrealizzabile nella pratica; ad esempio, testare tutte le combinazioni per sommare due interi a 32 bit richiederebbe oltre 584 milioni di anni. La vera difficoltà ingegneristica sta quindi nel selezionare input "buoni", ovvero un sottoinsieme di dati con un'elevata probabilità di rivelare un difetto. In generale, individuare e rimuovere i difetti nelle fasi finali di sviluppo costa molto di più rispetto alle fasi iniziali.

16.2 Terminologia Fondamentale

È essenziale distinguere tre concetti legati in una precisa catena causale:

- **Errore (Error):** Lo sbaglio umano commesso dallo sviluppatore, che può essere logico, concettuale o di semplice battitura.
- **Difetto (Fault o Bug):** La traduzione dell'errore nel codice sorgente. Un fault può sopravvivere silente a lungo nel programma, finché non viene fornito un input che sollecita quell'esatta istruzione.
- **Malfunzionamento (Failure):** Il comportamento anomalo o il crash del sistema che diventa visibile all'utente durante l'esecuzione.

Il processo che segue l'individuazione di un failure per trovare e correggere il fault associato si definisce **Debugging**, e si divide in localizzazione (fault localization) e rimozione (fault removal).

16.3 Tecniche di Testing

- **White-box (Strutturale):** Si basa sulla conoscenza esplicita della struttura interna e del codice sorgente del sistema, con l'obiettivo di coprire tutte le istruzioni logiche.
- **Black-box (Funzionale):** Non richiede la conoscenza del codice interno; gli input e gli output attesi vengono generati unicamente a partire dai documenti di specifica e dai requisiti.

16.4 Livelli di Testing

Il testing viene applicato a diversi livelli gerarchici man mano che il sistema viene costruito:

- **Testing di Unità:** La singola componente (es. una classe o una funzione) viene testata dallo sviluppatore in modo isolato dal resto del sistema.
- **Testing di Integrazione:** Verifica il corretto scambio di informazioni e il rispetto dei protocolli di comunicazione tra più moduli software assemblati assieme.
- **Testing di Sistema:** Valuta globalmente se l'intero sistema soddisfa i requisiti funzionali e i bisogni non funzionali (come usabilità, sicurezza, performance).
- **Testing di Regressione:** Consiste nella riesecuzione di una batteria di test ogni volta che si effettua una modifica al codice (es. per inserire una patch), per assicurarsi che non siano stati introdotti effetti collaterali (*side effect*) in funzionalità che prima operavano correttamente.

16.5 Testcase e Automazione con JUnit

Un **Testcase** è un insieme rigoroso composto da input, precondizioni di esecuzione e risultati attesi, mentre una **Testsuite** è una collezione che raggruppa più testcase. In Java, il testing automatizzato viene eseguito sfruttando il framework **JUnit 5**. Questo strumento permette di sostituire l'uso rudimentale e manuale del costrutto `main()` avvalendosi di classi di test dedicate. Elementi chiave dell'uso di JUnit includono:

- **Asserzioni:** Affermazioni logiche (es. `assertTrue()`, `assertEquals()`) che verificano automaticamente se il risultato ottenuto coincide con quello atteso. Se un'asserzione risulta falsa si verifica un *Failure*; se invece si manifesta un'eccezione a run-time nel codice sotto test, si parla di *Error*.
- **Isolamento e Coesione:** Tramite annotazioni specifiche come `@BeforeEach` e `@AfterEach`, il framework permette di eseguire logiche di inizializzazione e distruzione degli oggetti prima e dopo ogni singolo metodo di test, garantendo così che lo stato di partenza sia sempre neutro e identico.